МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №3

з курсу: „Системи баз даних”

виконав студент 3-го курсу
групи ДА-11
Сидько Н.С.

КИЇВ
2023

Системи графових баз даних

Порівнюючи з СБД інших типів, оброблення запиту до графової СБД потребує перегляду взаємозв’язків, що з’єднують вузли графа [19; 21; 22]. Графові бази даних застосовують для побудови соціальних сервісів, систем рекомендацій, автентифікації доступу до вебресурсів тощо.

Встановлення СБД Neo4j

Для встановлення цієї СБД, як контейнера у Docker пропишемо в консолі:

```
docker run --name some-neo4j -p 7474:7474 -p 7687:7687 -d neo4j
```

Тоді можна побачити встановлений сбд neo4j

```
7d97e254a046: Pull complete
732d09690fed: Pull complete
e8cba66f5b65: Pull complete
9e41d761a8cf: Pull complete
33a66ada74dc: Pull complete
Digest: sha256:eb36a96ef2d67050c92ca53109f10e0f24ac920971e58cdd0bf81aaec84a02e9
Status: Downloaded newer image for neo4j:latest
0d32746c5ce687cae5ccfbf930580b0e71dc8b634b80a44d90a83db7dcae5f4d
```

Відкривши Docker можна побачити встановлений some-neo4j

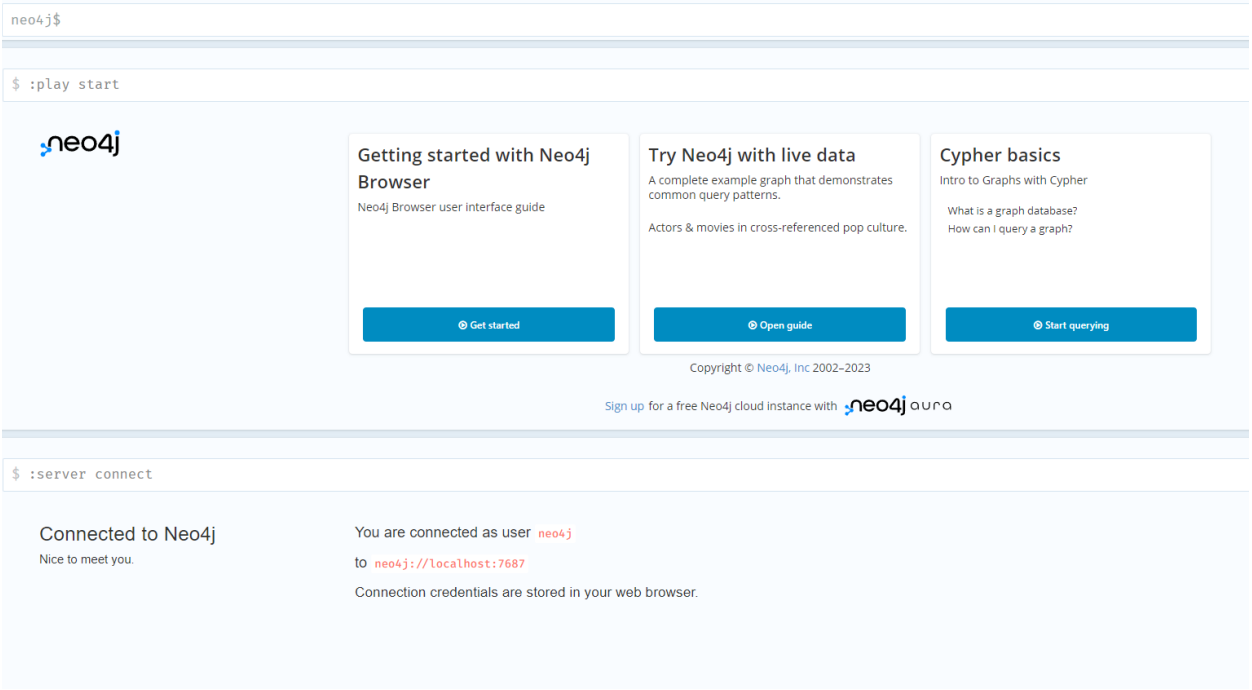
	Name	Image	Status	CPU (%)	Port(s)	Last started
☐	some-neo4j 8f360ca02f69	neo4j	Running	0.76%	7474:7474 Show all ports (2	8 minutes ago

Запустимо контейнер, та введемо команди **neo4j start** та **neo4j status**



В neo4j реалізовано веб інтерфейс, для його відкриття перейдемо за посиланням де ми можемо побичити сам інтерфейс та зареєструватись.

Після реєстрації отримуємо:



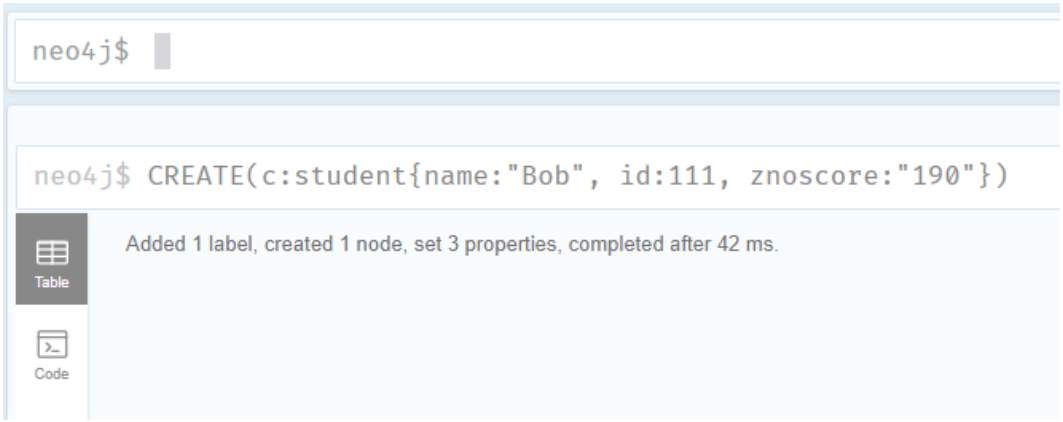
Мова запитів Cypher

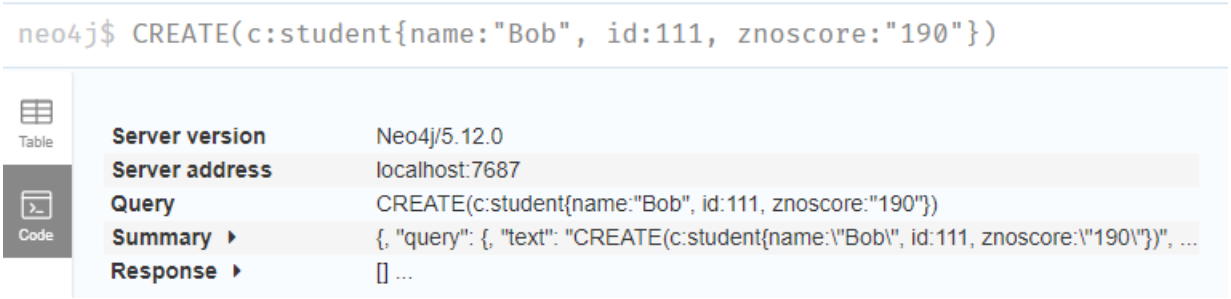
Cypher — мова запитів до графової СБД. Якщо порівняти з теорією графів, вузли графа називаються «вершинами», взаємозв’язки між вузлами — «дугами».
Запити до графа формуються за шаблоном:
$MATCH [some set of nodes and/or relationships]
WHERE [some set of properties holds]
RETURN [some set of results captured by the MATCH and WHERE clauses]

CRUD запити до БД

У мові Cypher оператори **MATCH** і **CREATE** створюють нові вузли / взаємозв’язки; оператор **SET** оновлює значення властивостей наявних вузлів / взаємозв’язків графа.

Створимо вузол с типом «студент» і властивостями «ім’я» (Bob), «id»(111), «бал зно» (190)



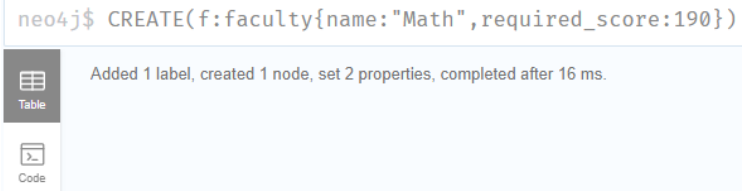


В інтерфейсі виведено повідомлення СБД, у верхній частині розміщується запит Cypher. Вкладка Table зчитує вузли та/або взаємозв’язки, створені запитом Cypher. Вкладка Code містить системну інформацію щодо запиту.

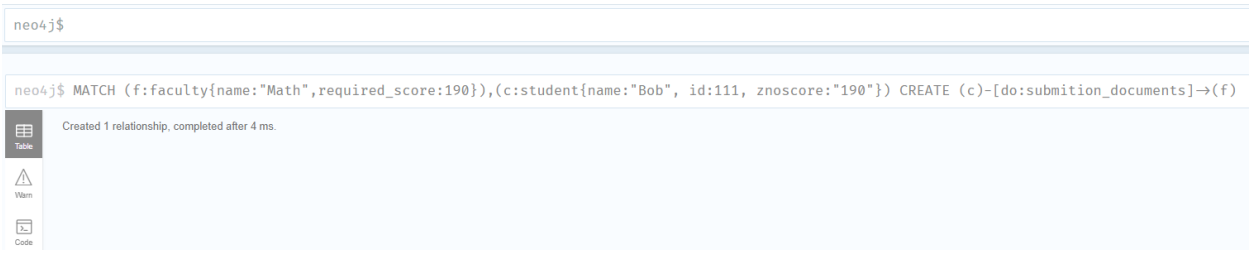
За допомогою оператора Match зчитаємо вузли графа за умовою запиту.



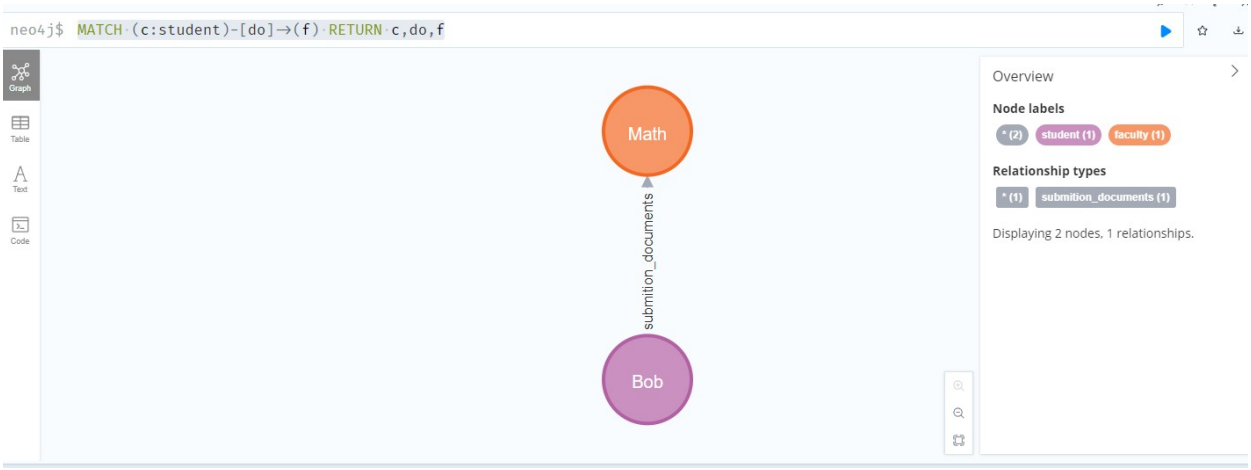
Створимо новий вузол з типом факультет



Створимо зв’язок з типом submition_document для вузлів з типами faculty і student:



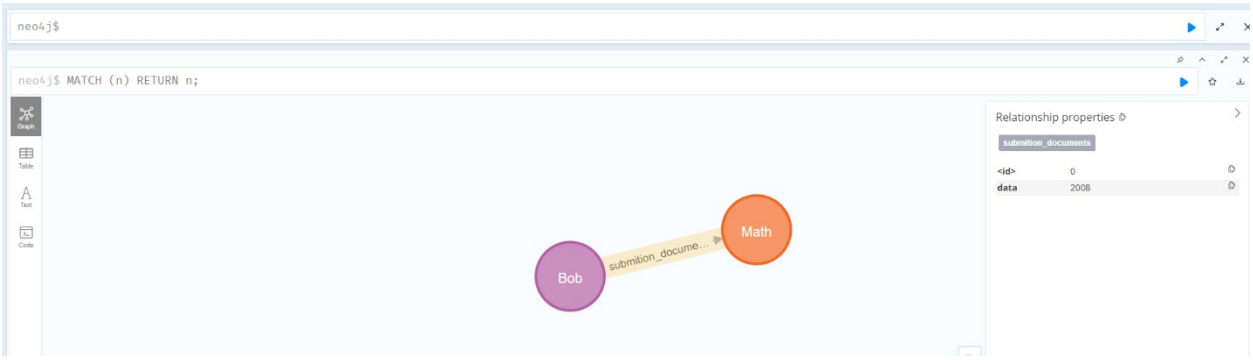
Оператор **CREATE** створює зв’язок з типом deposit_operation для вузлів за умови відповідності **MATCH** значенню властивості name. Графічно воно виглядатиме так:



Взаємозв’язки, як і вузли, можуть містити властивості. Доповнимо зв’язок № 0 властивістю «дата подачі»:

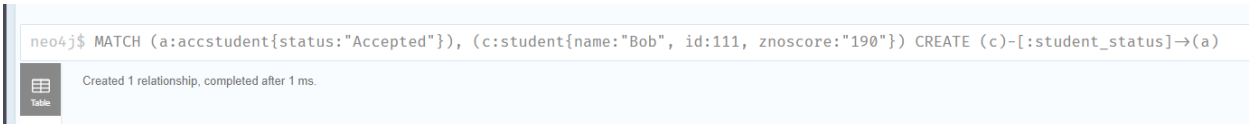


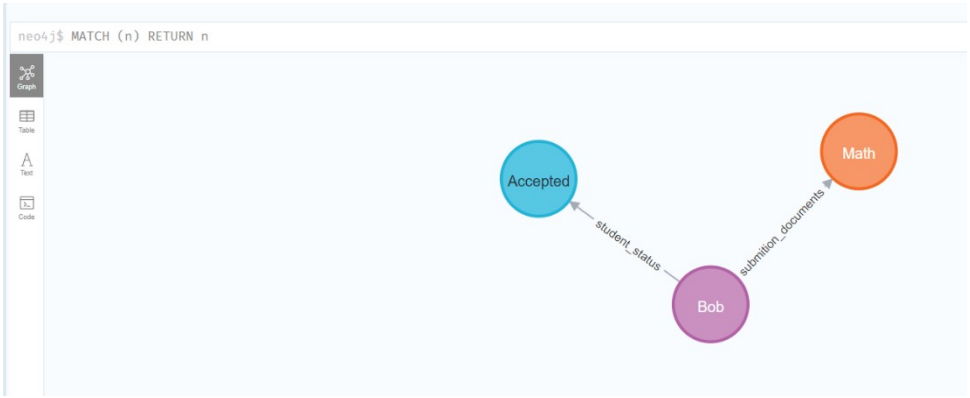
При зчитуванні всього графу запитом MATCH (n) RETURN n; і курсором переміститися за взаємозв’язком між вузлами, з’явиться значення 2008 (рік подачі документа), яке є властивістю взаємозв’язку deposit_operation:



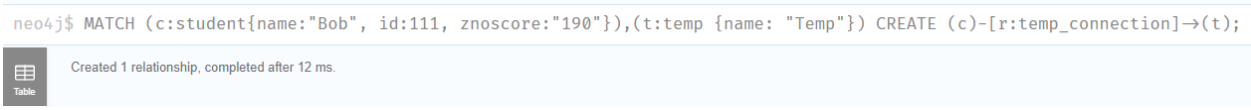
Зазначення властивостей допускається і в запиті створення взаємозв’язку.

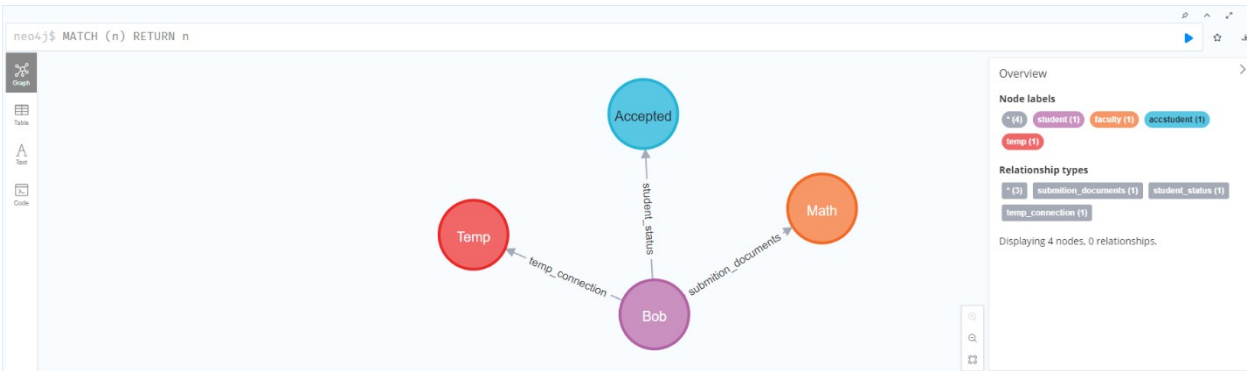
Створимо новий вузол «статус студента» з властивістю «статус» (студент може бути прийнятий / не прийнятий)





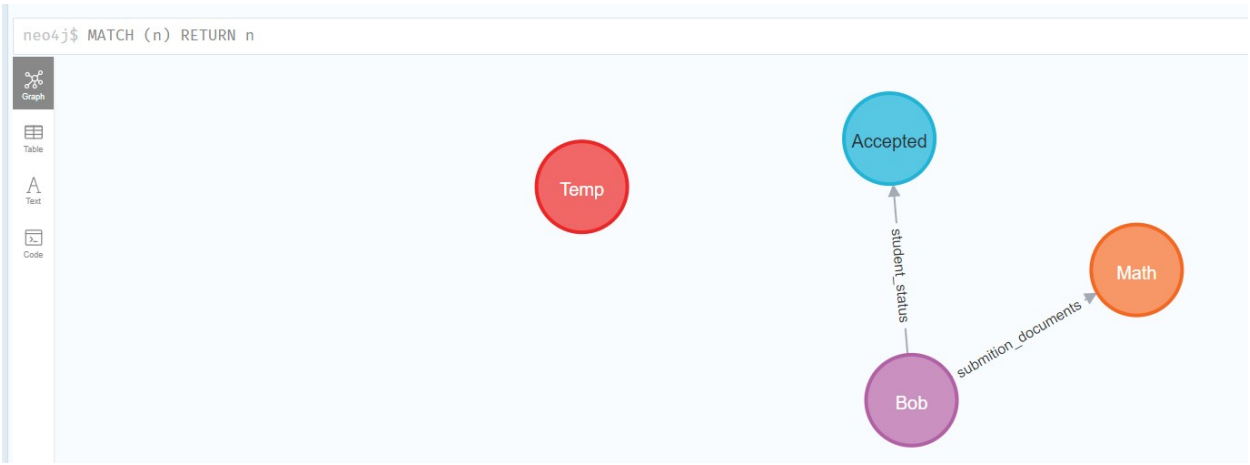
Наявний граф містить вузли трьох типів: «студент», «факультет» і «статус студента». Тепер створимо тестовий вузол і зв’язок для подальшого його видалення :



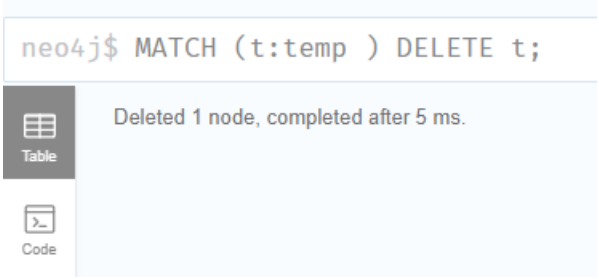


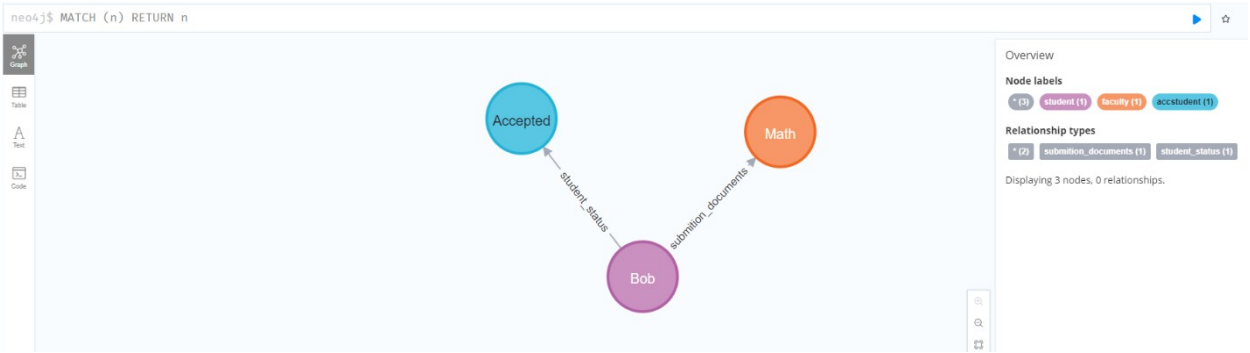
Видалимо тестовий зв’язок за вказаним типом:





Тепер видалимо саму ноду:





Як бачимо граф повернувся до вигляду до створення тестових елементів. Крім того є також шаблон запиту Cypher для видалення всіх вузлів і взаємозв’язків, тобто всього графа.
MATCH (n) OPTIONAL MATCH (n)-[r]- () DELETE n, r;

Добудуємо граф студентів. Додамо ще три вузли, два з яких будут студентами, а третій викладачем.

```
neo4j$ CREATE (teach:teacher {name:"Petro"});
```

Table

Code

Added 1 label, created 1 node, set 1 property, completed after 6 ms.

```
neo4j$ CREATE (c:student{name:"Rick", id:112, znoscore:"195"});
```

Table

Code

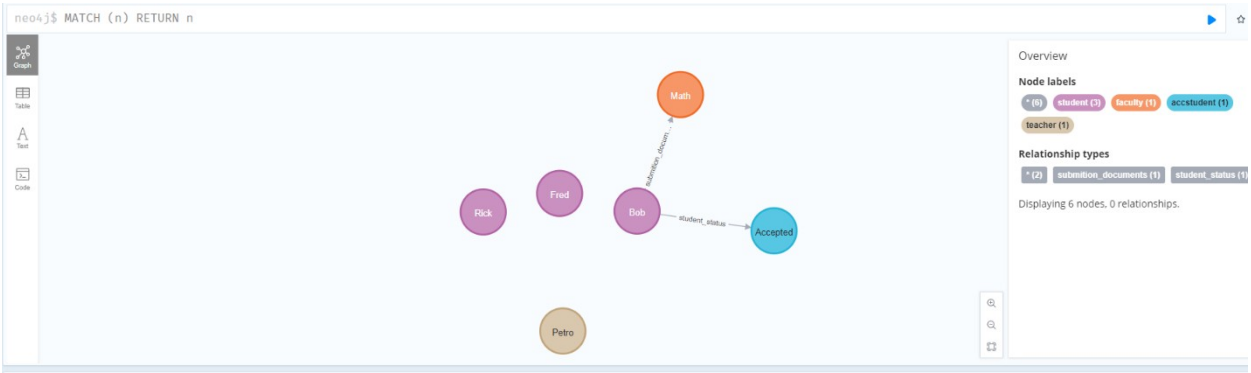
Added 1 label, created 1 node, set 3 properties, completed after 5 ms.

```
neo4j$ CREATE (c:student{name:"Fred", id:113, znoscore:"200"});
```

Table

Code

Added 1 label, created 1 node, set 3 properties, completed after 5 ms.



Додамо певні зв’язки студентів з викладачем.

```
neo4j$ MATCH (c:student {name:"Bob", id:111, znoscore:"190"}), (teach:teacher{name:"Petro"}) CREATE (teach)-[tch:teacher]→(c);
```

Table

Created 1 relationship, completed after 21 ms.

```
neo4j$ MATCH (c:student {name:"Rick", id:112, znoscore:"195"}), (teach:teacher{name:"Petro"}) CREATE (teach)-[tch:teacher]→(c);
```

Table

Created 1 relationship, completed after 9 ms.

```
neo4j$ MATCH (c:student {name:"Fred", id:113, znoscore:"200"}), (teach:teacher{name:"Petro"}) CREATE (teach)-[tch:teacher]→(c);
```

Table

Created 1 relationship, completed after 8 ms.

А також додамо зв’язок того, що вони прийняті до інституту

```
neo4j$ MATCH (c:student {name:"Bob", id:111, znoscore:"190"}), (a:accstudent {status:"Accepted"}) CREATE (c)-[stat:student_status]→(a)
```

Created 1 relationship, completed after 1 ms.

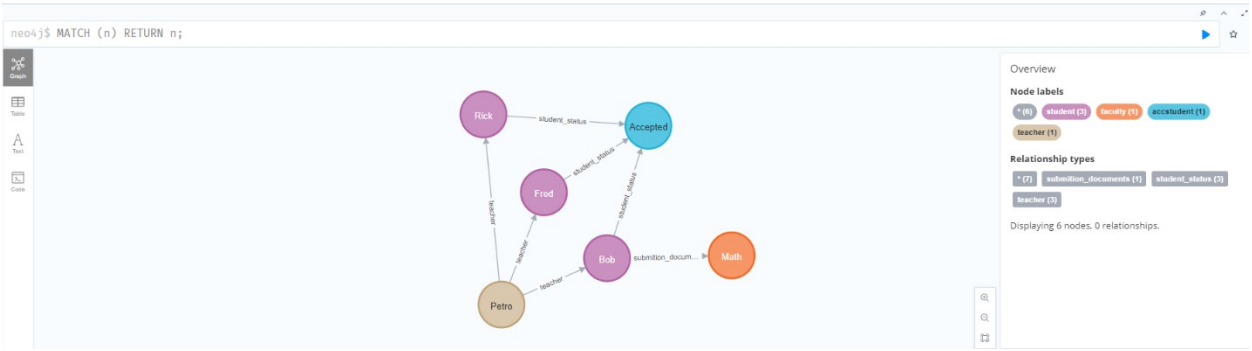
```
neo4j$ MATCH (c:student {name:"Fred", id:113, znoscore:"200"}), (a:accstudent {status:"Accepted"}) CREATE (c)-[stat:student_status]→(a);
```

Created 1 relationship, completed after 8 ms.

```
neo4j$ MATCH (c:student {name:"Rick", id:112, znoscore:"195"}), (a:accstudent {status:"Accepted"}) CREATE (c)-[stat:student_status]→(a)
```

Created 1 relationship, completed in less than 1 ms.

Тоді отримуємо



Приклад соціальної мережі

Створимо людей які ще не є студентами КПІ, але дізнавались у викладачів на заняттях з підготовки до ЗНО щодо нього. Для цього створимо ще одного викладача, який буде мати взаємозв’язок з іншими викладачами оскільки вони працюють в одному інституті.
Школярі:

```
neo4j$ CREATE (sch:schoolboy {name:"Misha"});
```

Added 1 label, created 1 node, set 1 property, completed after 4 ms.

```
neo4j$ CREATE (sch:schoolboy {name:"Masha"});
```

Added 1 label, created 1 node, set 1 property, completed after 4 ms.

```
neo4j$ CREATE (sch:schoolboy {name:"Valera"});
```

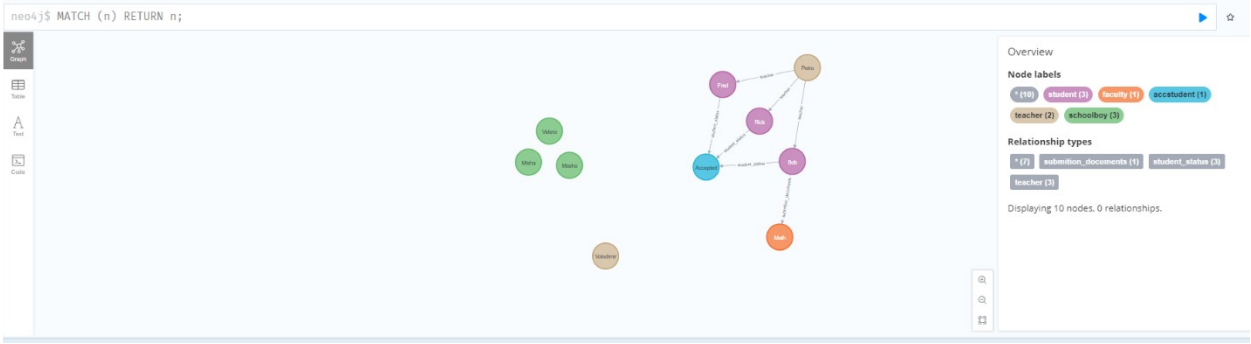
Added 1 label, created 1 node, set 1 property, completed after 31 ms.

Викладач:

```
neo4j$ CREATE (teach:teacher {name:"Volodimir"});
```

Added 1 label, created 1 node, set 1 property, completed after 3 ms.

Тоді маємо наступне:



Створимо зв’язок між школярами, наприклад вони друзі:

```
neo4j$ MATCH (sc1:schoolboy {name:"Masha"}), (sc2:schoolboy {name:"Valera"}) CREATE (sc1)-[scl:friends]→(sc2);
```

Created 1 relationship, completed after 19 ms.

```
neo4j$ MATCH (sc2:schoolboy {name:"Valera"}), (sc3:schoolboy {name:"Misha"}) CREATE (sc2)-[scl:friends]→(sc3);
```

Created 1 relationship, completed after 7 ms.

Тепер створимо зв’язки між школярами та студентами, а також викладачами внз.

Викладач зі школярем

```
neo4j$ MATCH (sc3:schoolboy {name:"Misha"}), (teach:teacher {name:"Volodimir"}) CREATE (sc3)-[tch:teacher]→(teach);
```

Created 1 relationship, completed after 6 ms.

Викладач з викладачем

```
neo4j$ MATCH (teach2:teacher {name:"Petro"}), (teach1:teacher {name:"Volodimir"}) CREATE (teach2)-[tch:collegues]→(teach1);
```

Created 1 relationship, completed after 6 ms.

Created 1 relationship, completed after 6 ms.

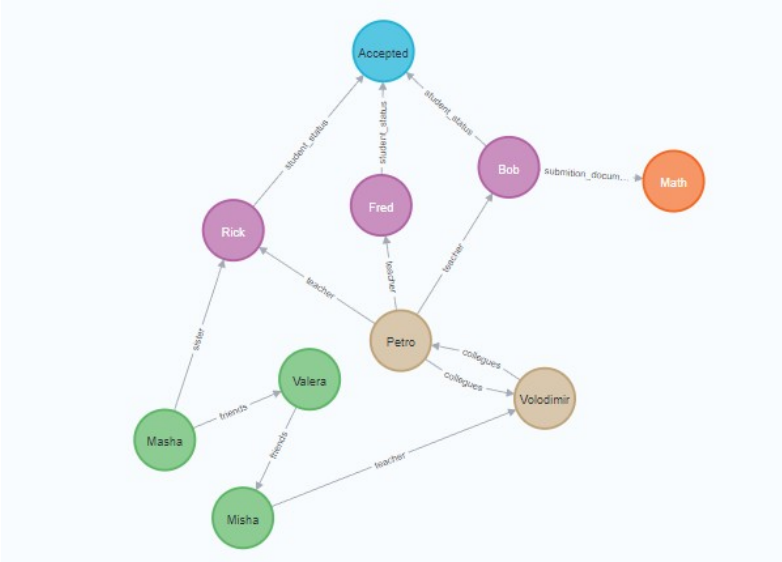
```
neo4j$ MATCH (teach1:teacher {name:"Volodimir"}), (teach2:teacher {name:"Petro"}) CREATE (teach1)-[tch:collegues]→(teach2);
```

Created 1 relationship, completed after 20 ms.

Студент зі школярем

```
neo4j$ MATCH (sc3:schoolboy {name:"Masha"}), (c:student {name:"Rick", id:112, znoscore:"195"}) CREATE (sc3)-[sis:sister]→(c);
```

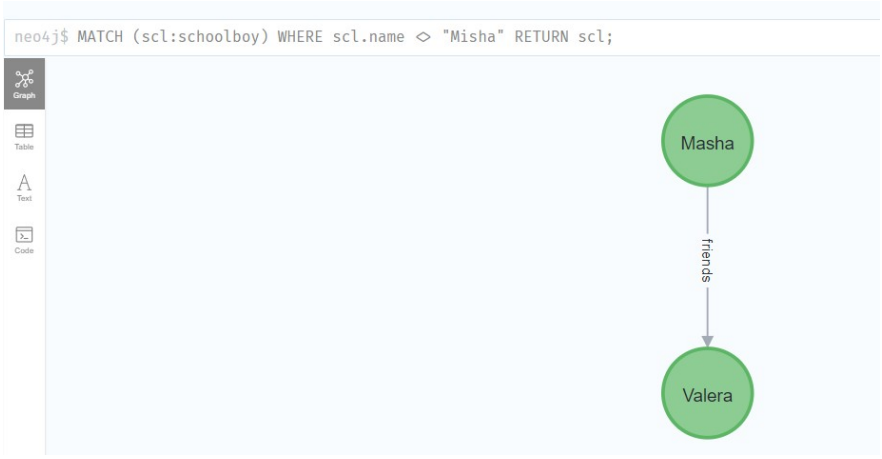
Created 1 relationship, completed after 18 ms.



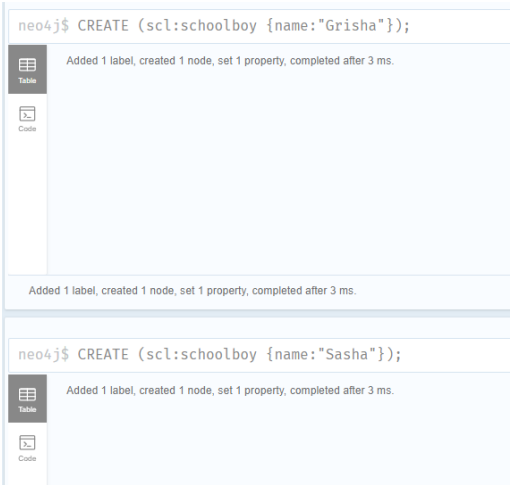
Для пошуку всіх взаємозв’язків певного вузла існує оператор «-->». Спробуємо зчитати всі вузли, взаємозв’язані з Masha:

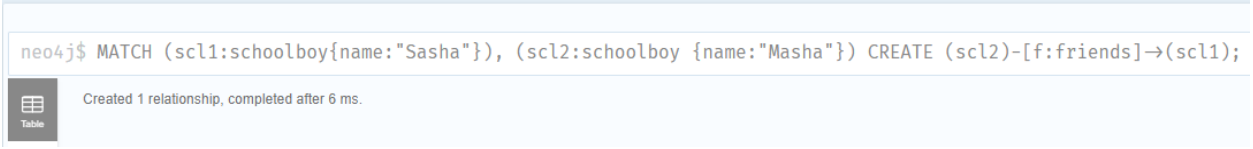


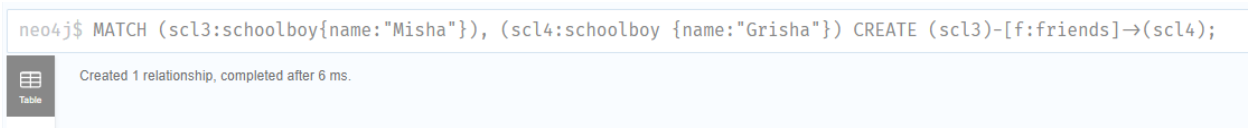
Також зчитаємо вузли schoolboy, за винятком імені Misha:



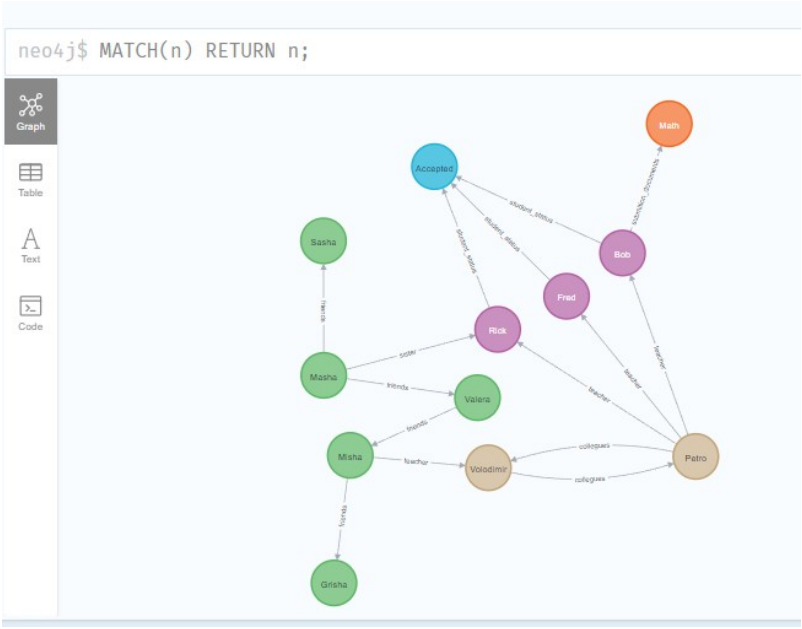
Створимо деякі вузли, які не зв’язані безпосередньо з Valera, а саме: Sasha з Masha, Misha з Grisha:



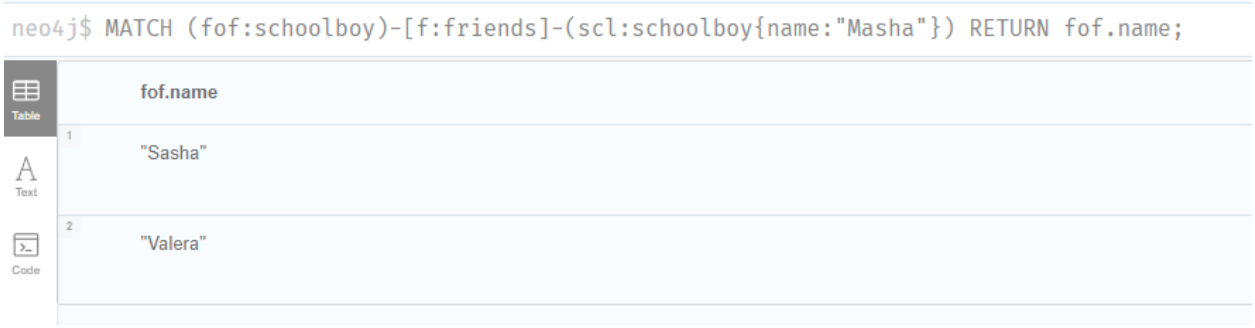




В результаті отримуємо:



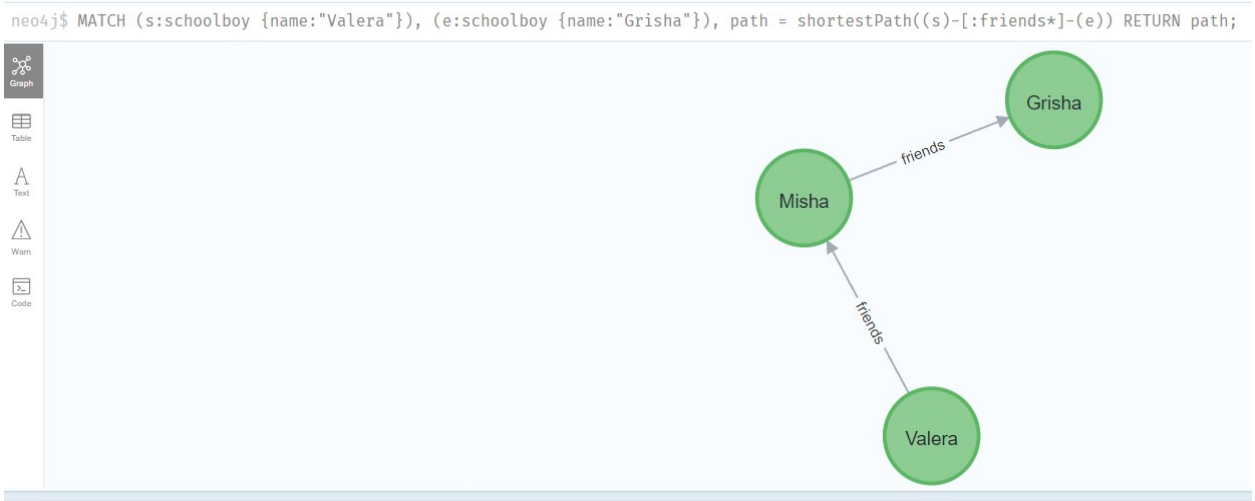
Знайдемо вузли, взаємозв’язані з вузлом графа:



Бачимо що у нас показує 2 вузли, а отже робимо висновок що все вірно.

Вкладені взаємозв’язки

Зчитаємо вкладені взаємозв’язки між вузлами з типом schoolboy, застосувавши алгоритм пошуку shortestPath (є також allPaths,dijkstra…) :

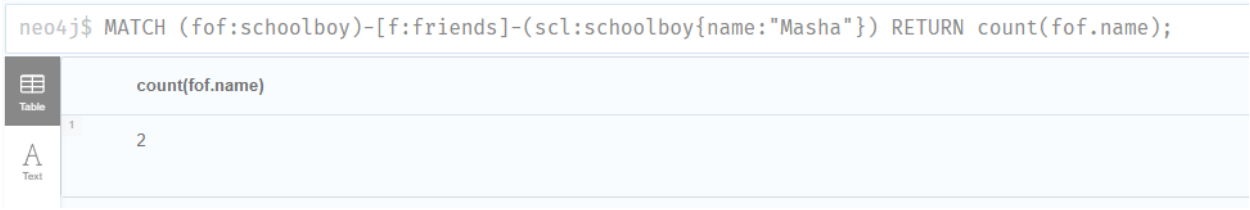


Агрегатні функції

В мові Cypher міститься такі агрегатні функції:

- обчислення кількості вузлів(взаємозв’язків) в графі — **count**(),
- виконання фільтрації — **filter**().

Зчитаємо кількість наявних у графі взаємозв’язків з типом friends



Створення індексів

Створимо індекс за типом вузла та властивістю

Створимо умову унікальності для ID студента та спробуємо додати ще одного із номером телефону 111:

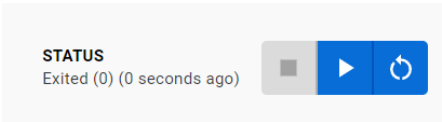


Як бачимо умова унікальності виконується. Тепер спробуємо прибрати цю умову:





Вимикаємо контейнер:



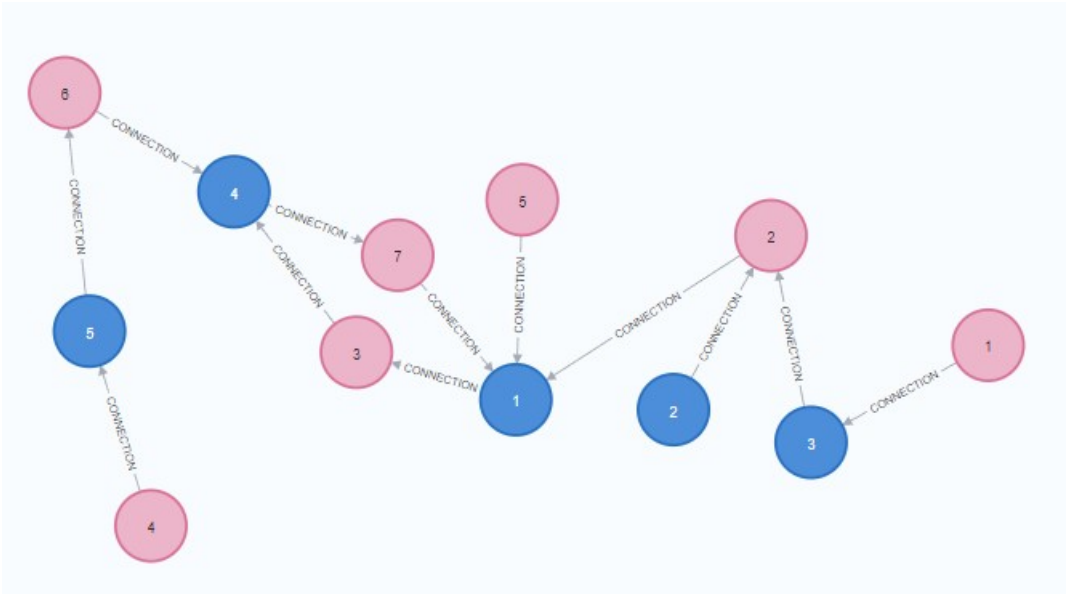
Практичне завдання

1.Встановіть бібліотеку мови програмування на ваш вибір і програмно створіть граф — сім вузлів (тип № 1), які взаємозв’язані з п’ятьма вузлами (тип № 2).

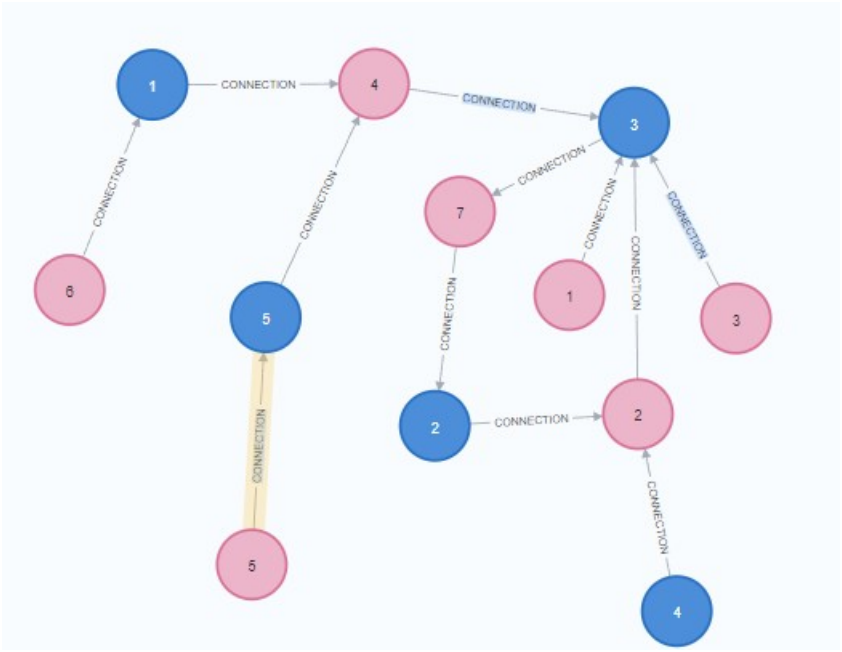
```
import neo4j
import random

#Підключення до бази даних Neo4j
driver = neo4j.GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j",
"duhegepug123"))
with driver.session() as session:
    #Очищення минулих даних
    session.run("MATCH (n) DETACH DELETE n")
    # Створення семи вузлів типу 1
    for i in range(7):
        session.run("CREATE (n:Node1 {id: $id})", {"id": i + 1})
    # Створення п'яти вузлів типу 2
    for i in range(5):
        session.run("CREATE (n:Node2 {id: $id})", {"id": i + 1})
    # Створення зв'язків між вузлами
    for i in range(7):
        randnode2 = random.randint(1,5)
        session.run("MATCH (n1:Node1 {id: $id1}), (n2:Node2 {id: $id2}) CREATE (n1)-
[:CONNECTION]->(n2)",{"id1": i + 1, "id2": randnode2})
    for j in range(5):
        randnode1 = random.randint(1,7)
        session.run("MATCH (n1:Node1 {id: $id1}), (n2:Node2 {id: $id2}) CREATE (n2)-
[:CONNECTION]->(n1)", {"id1": randnode1, "id2": j + 1})
driver.close()
```

Результат роботи коду :



2. Застосовуючи встановлену бібліотеку API, програмно реалізуйте функцію зчитування в консолі ланцюжків (за зменшенням відстані) для всіх допустимих поєднань кінцевих вузлів зі спільним типом.




```
Ланцюжок 1го типу:
Ланцюжок:
{'id': 1}->{'id': 3}->{'id': 7}->{'id': 2}->{'id': 2}-> Кінець
Ланцюжок:
{'id': 1}->{'id': 3}->{'id': 7}-> Кінець
Ланцюжок:
{'id': 2}->{'id': 3}->{'id': 7}-> Кінець
Ланцюжок:
{'id': 3}->{'id': 3}->{'id': 7}-> Кінець
Ланцюжок:
{'id': 4}->{'id': 3}->{'id': 7}-> Кінець
Ланцюжок:
{'id': 6}->{'id': 1}->{'id': 4}->{'id': 3}->{'id': 7}-> Кінець
Ланцюжок:
{'id': 5}->{'id': 5}->{'id': 4}->{'id': 3}->{'id': 7}-> Кінець

Ланцюжок 2го типу:
Ланцюжок:
{'id': 1}->{'id': 4}->{'id': 3}-> Кінець
Ланцюжок:
{'id': 1}->{'id': 4}->{'id': 3}->{'id': 7}->{'id': 2}-> Кінець
Ланцюжок:
{'id': 1}->{'id': 4}->{'id': 3}->{'id': 7}->{'id': 2}->{'id': 2}->{'id': 3}-> Кінець
Ланцюжок:
{'id': 2}->{'id': 2}->{'id': 3}-> Кінець
```

Висновок

У ході виконання лабораторної роботи №3 були набуті навички роботи з графовою СБД Neo4j. Для виконання CRUD операцій був використаний веб інтерфейс. Взаємодія із БД здійснювалась за допомогою мови запитів Cypher. Також було створено програмно програмно, користуючись мовою Python та бібліотекою neo4j для роботи із СБД.